Event Sync AI: An Agentic Event Volunteer Coordination Platform

Dr. Neelima Gogineni
Department of Computer Science and Engineering
Koneru Lakshmaiah Education Foundation
Hyderabad, India
[drneelima@klh.edu.in]

R. Ananya Remani
Department of Computer Science and Engineering
Koneru Lakshmaiah Education Foundation
Hyderabad, India
[2420030086@klh.edu.in]

B. Nageshwari
Department of Computer Science and Engineering
Koneru Lakshmaiah Education Foundation
Hyderabad, India
[2420030426@klh.edu.in]

P. Sri Maha Lakshmi
Department of Computer Science and Engineering
Koneru Lakshmaiah Education Foundation
Hyderabad, India
[2420030313@klh.edu.in]

Abstract— Event management activities often require coordinating large numbers of volunteers across different tasks and responsibilities. Traditional volunteer coordination methods rely on spreadsheets, manual communication and human judgement, making it difficult to assign volunteers based on their skills, availability, experience and current workload. These methods also create challenges when volunteers become unavailable at the last minute.

EventSync AI is an intelligent event volunteer coordination platform designed to simplify volunteer management using Service-Oriented Architecture principles. The platform consists of modules for event management, volunteer management, task assignment, dynamic reassignment and notifications. An intelligent allocation mechanism evaluates volunteer skills, experience and workload to identify suitable candidates for available tasks. When a volunteer becomes unavailable, the system supports reassignment of the task to another eligible volunteer.

The application uses React.js for the frontend and Java Spring Boot for backend development, with a relational database for storing event, volunteer and assignment information. JWT-based authentication and role-based access control are incorporated to provide secure access to the platform. The proposed system aims to reduce manual coordination, improve volunteer-task matching, support faster reassignment and provide a more organised approach to event volunteer management.

Keywords— EventSync AI, Artificial Intelligence, Volunteer Coordination, Service-Oriented Architecture, Intelligent Task Allocation, Dynamic Reassignment, Spring Boot, React.js, JWT Authentication, Role-Based Access Control.

I. INTRODUCTION

Events such as campus programmes, public gatherings, charity activities and community initiatives depend on volunteers to complete tasks ranging from registration and logistics to safety support and participant coordination. The success of these activities depends not only on the number of volunteers available but also on whether volunteers are assigned to tasks that suit their skills, experience and availability. Manual coordination can become difficult as the number of events, tasks and volunteers increases.

Traditional methods such as spreadsheets, messaging applications and telephone calls can provide basic coordination, but information may be distributed across multiple channels. Organizers may need to repeatedly check availability, identify suitable replacements and communicate assignment changes. Research on volunteer scheduling has shown the value of decision-support tools that consider constraints and volunteer preferences [1]. Later work has examined task assignment algorithms, volunteer retention and the effects of worker-task matching [2]–[8].

EventSync AI is designed to bring event information, volunteer profiles, tasks, assignments and notifications into a common platform. The coordination component ranks eligible volunteers using a deterministic score based on skill overlap, experience and current workload. It also supports reassignment when a volunteer becomes unavailable. The current repository implements these functions through a React/Vite frontend and a Spring Boot REST API. The backend is organized into domain entities and repository/controller components within one deployable application. The broader project design follows service-oriented principles and can be extended into separately deployed services.

The main contributions of this work are: (1) a unified event and volunteer workflow; (2) an explainable rule-based volunteer-task matching mechanism; (3) reassignment handling triggered by a volunteer availability change; and (4) role-based access to coordination functions. The work is a software prototype and does not claim a measured improvement in event outcomes without a controlled evaluation.

II. RELATED WORK

The literature is reviewed under three themes: volunteer scheduling and retention, skill-aware task allocation, and service architecture and security.

A. Volunteer Scheduling and Retention

Gordon and Erkut developed a decision-support tool for scheduling volunteers at the Edmonton Folk Festival. Their approach generated shift schedules while considering operational constraints and volunteer preferences [1]. Kaur et al. proposed an integer linear programming model for assigning volunteers and employees to tasks while accounting for task urgency, organizational preferences, training requirements and the possible effect of assignments on volunteer retention [6]. Kaur and Pazour later proposed an optimization framework that creates task recommendation menus and volunteer groups, with the goal of balancing organizational requirements and volunteer autonomy [7]. These studies indicate that effective coordination should consider both operational suitability and the volunteer experience.

B. Skill-Aware Task Allocation and Reassignment

Ho and Vaughan studied online assignment of heterogeneous tasks to workers with different skill levels, presenting an algorithm that accounts for uncertainty in worker capabilities [2]. Assadi et al. examined online assignment of heterogeneous tasks under feasibility and budget constraints [3]. Bernstein and colleagues investigated routing tasks to workers based on cognitive abilities, showing the relevance of matching task requirements with worker characteristics [4]. Kumai et al. considered skill balance and worker stress when forming groups for task streams [5]. Matinrad and Granberg studied volunteer dispatch for emergency response, incorporating uncertainty and comparing assignment strategies [8]. Together, these works motivate matching based on capabilities and operational constraints, while also showing that availability and reassignment are important in dynamic settings.

C. Service Architecture and Access Control

Di Francesco, Lago and Malavolta mapped research on architecting with microservices and discussed the state of the art, architectural approaches and research gaps [9]. Their work provides a basis for separating business capabilities into services with defined communication boundaries. Le et al. examined the extraction and validation of role-based access-control policies in web applications, emphasizing the importance of checking whether implemented permissions match intended access rules [10]. These findings support a modular architecture and explicit role-based authorization for an event coordination platform.

D. Summary

Existing studies address volunteer scheduling, task allocation, worker suitability, volunteer retention, microservice architecture and access-control validation. However, these areas are often investigated

separately. EventSync AI combines event and task records, volunteer profiles, rule-based matching, availability-triggered reassignment and notifications in one application workflow. This integration is the practical focus of the proposed platform; it is not presented as proof that no comparable system exists.

III. RESEARCH GAP AND MOTIVATION

The reviewed work provides methods for volunteer scheduling and optimization [1], [6], [7], as well as algorithms for matching workers to heterogeneous tasks [2]–[5], [8]. Nevertheless, an event organizer using disconnected tools may still need to translate volunteer profiles and task requirements into assignments, monitor availability changes and communicate replacements manually. A further challenge is explainability: an organizer should be able to understand why a volunteer was selected and intervene when operational judgement is required.

The research motivation is to integrate these coordination activities into a single event-focused workflow. EventSync AI uses a transparent scoring rule rather than a trained machine-learning model. It prioritizes required-skill overlap, adds an experience contribution and penalizes higher current workload. When an assigned volunteer becomes unavailable, the system attempts to select another available volunteer and records a notification. If no replacement is available, the task is reopened for coordinator attention.

The resulting research gap is therefore framed as an application-level integration problem: connecting skill-aware task selection, availability updates, workload awareness, reassignment and role-specific access in an event management platform. The system provides a working prototype for this workflow, while large-scale optimization, fairness analysis and field evaluation remain future research activities.

IV. METHODOLOGY

The development follows a requirements-to-implementation process comprising domain modelling, API development, assignment logic, security configuration, persistence and deployment preparation.

A. System Design

The application has a React/Vite frontend and a Java 17 Spring Boot backend. The backend exposes REST endpoints for events, volunteers, tasks, assignments, notifications, dashboard information and authentication. Domain entities represent Event, Volunteer, EventTask, Assignment, Notification and AppUser. Repository interfaces provide persistence operations. The current code is packaged as one Spring Boot application; the service-oriented design separates responsibilities logically, while independent service deployment, Eureka registration and Spring Cloud Gateway remain planned architectural extensions.

B. Volunteer and Task Modelling

A volunteer record stores name, email, skills, experience level, availability and a maximum workload. A task record stores its title, event association, description, required skills, workload and status. An assignment connects an event, task and volunteer and tracks its status and assignment time. This representation allows the matching component to compare task requirements with volunteer profiles and current assignments.

C. Rule-Based AI Allocation

The auto-assignment operation considers open tasks and processes higher-workload tasks first. For each task without an active assignment, it considers volunteers marked available. The score implemented in the backend is: Score = 100 × SkillMatches + 5 × ExperienceLevel − 10 × CurrentLoad. Experience is mapped to an ordinal value: advanced = 3, intermediate = 2 and beginner/default = 1. Current load is calculated from the workload of active assignments. The selection first prefers candidates within the configured workload limit; if none meet that condition, it falls back to the available pool and selects the highest-scoring candidate. This deterministic method is explainable and should be reviewed by an organizer. It is not a trained machine-learning model.

D. Dynamic Reassignment and Notifications

When a volunteer's availability changes from true to false, the backend retrieves that volunteer's active assignments. Each assignment is marked as reassigned, and the system searches for an available replacement while excluding the unavailable volunteer. If a replacement is found, a new assignment is created and a reassignment notification is stored. If no suitable replacement is found, the task is returned to the open state and a warning notification indicates that coordinator action is required.

E. Security and Data Management

The application uses Spring Security with stateless bearer-token authentication. Passwords are encoded using BCrypt. JWT claims identify the user and role, and endpoint authorization distinguishes ADMIN, ORGANIZER, TEAM_LEAD and VOLUNTEER permissions. Public registration is restricted to volunteer accounts. The development profile uses an H2 database, while the containerized production profile is configured for PostgreSQL with Flyway migrations. Docker Compose defines database, backend and frontend containers, persistent PostgreSQL storage and health checks.

F. Testing and Evaluation

The uploaded repository includes token-service unit tests covering successful token verification, rejection of modified tokens and rejection of malformed tokens. The project documentation also identifies additional checks required before production use, including endpoint authorization tests, integration and API contract tests, security scanning, load testing, backup restoration and monitoring. These are treated as validation requirements, not as completed experimental results. A future evaluation should measure assignment suitability, reassignment success, task completion, response time and workload distribution using representative event data.

V. RESULTS AND DISCUSSION

Inspection of the submitted source code confirms implementation of the core event coordination workflow. The API supports event, volunteer and task creation and updates, assignment listing, automatic assignment, assignment completion, notification retrieval and volunteer availability updates. The dashboard aggregates event, volunteer, task, assignment and notification counts, together with volunteer workload information.

A. Functional Coordination

The automatic assignment endpoint filters open tasks, sorts them by workload in descending order and skips tasks that already have an active assignment. It then selects a volunteer using the rule-based matching score. Tasks for which no available volunteer can be selected are returned in the skipped-task list. This provides an organizer with a clear indication of work that still requires attention.

B. Availability-Driven Reassignment

The volunteer update and availability endpoints invoke reassignment when a volunteer changes from available to unavailable. Existing active assignments are marked REASSIGNED. A replacement assignment is created when a candidate is available; otherwise, the task is reopened and a warning is recorded. This behavior addresses short-notice unavailability at the application workflow level. The implementation does not establish that a replacement will always be found.

C. Security and Persistence

Role checks are configured for administrative, organizer, team-lead and volunteer operations. The backend uses BCrypt password encoding and signed expiring bearer tokens. PostgreSQL persistence, Flyway migration support and container health checks are included in the deployment baseline. The repository documentation also identifies remaining production requirements, including HTTPS, managed secrets, stronger token/session controls, audit logging, comprehensive security testing and operational monitoring.

D. Limitations

The submitted codebase is currently a single Spring Boot backend rather than a set of independently deployed microservices. The repository does not establish implemented Eureka service discovery, Spring Cloud Gateway routing, Kafka messaging or external email/SMS delivery. The frontend uses periodic API polling for cross-user updates rather than push-based real-time delivery. The recommender is deterministic and rule-based, and no controlled benchmark or field study is included. Accordingly, this paper reports implemented functions and architecture boundaries without claiming measured performance gains or production certification.

VI. CONCLUSION AND FUTURE SCOPE

EventSync AI presents a unified platform for managing events, volunteers, tasks, assignments and coordination notifications. Its rule-based recommender uses skill overlap, experience and current workload to suggest volunteer-task matches. Availability-triggered reassignment provides a mechanism for responding to volunteer unavailability, while role-based authorization separates access for administrators, organizers, team leads and volunteers. The current implementation demonstrates these workflows through a React/Vite frontend and a Spring Boot REST backend.

Future work includes decomposing the modular backend into independently deployable Event, Volunteer, Task/Assignment and Notification services; adding Eureka service discovery and Spring Cloud Gateway; and introducing reliable asynchronous messaging where required. Further improvements may include preference-aware and fairness-aware allocation, historical reliability signals, workload forecasting, authenticated SSE/WebSocket updates, external notification providers, richer audit trails and comprehensive automated testing. Evaluation with real or carefully simulated event data is required to quantify assignment quality, reassignment effectiveness, volunteer satisfaction and system scalability.

ACKNOWLEDGMENT

The authors sincerely thank Dr. Neelima Gogineni, Department of Computer Science and Engineering, KLH CSE Bowrampet Campus, for her guidance and support throughout the project. The authors also thank the faculty members, department and institution for providing the academic environment and resources required for this work. Appreciation is extended to the researchers whose publications informed the literature review and design considerations.

REFERENCES

[1] L. Gordon and E. Erkut, “Improving volunteer scheduling for the Edmonton Folk Festival,” Interfaces, vol. 34, no. 5, pp. 367–376, 2004, doi: 10.1287/inte.1040.0097.

[2] C.-J. Ho and J. W. Vaughan, “Online task assignment in crowdsourcing markets,” in Proc. 26th AAAI Conf. Artificial Intelligence, 2012, pp. 45–51, doi: 10.1609/aaai.v26i1.8120.

[3] S. Assadi, J. Hsu, and S. Jabbari, “Online assignment of heterogeneous tasks in crowdsourcing markets,” Proc. AAAI Conf. Human Computation and Crowdsourcing, vol. 3, no. 1, pp. 12–21, 2015, doi: 10.1609/hcomp.v3i1.13236.

[4] J. Goncalves, M. Feldman, S. Hu, V. Kostakos, and A. Bernstein, “Task routing and assignment in crowdsourcing based on cognitive abilities,” in Proc. 26th Int. Conf. World Wide Web Companion, 2017, pp. 1023–1031, doi: 10.1145/3041021.3055128.

[5] K. Kumai et al., “Skill-and-stress-aware assignment of crowd-worker groups to task streams,” Proc. AAAI Conf. Human Computation and Crowdsourcing, vol. 6, no. 1, pp. 88–97, 2018, doi: 10.1609/hcomp.v6i1.13328.

[6] M. P. Kaur, S. Smith, J. A. Pazour, and A. Duque Schumacher, “Optimization of volunteer task assignments to improve volunteer retention and nonprofit organizational performance,” Socio-Economic Planning Sciences, vol. 84, Art. no. 101392, 2022, doi: 10.1016/j.seps.2022.101392.

[7] M. P. Kaur and J. Pazour, “An optimization framework to provide volunteers with task selection autonomy and group opportunities,” Socio-Economic Planning Sciences, vol. 96, Art. no. 102095, 2024, doi: 10.1016/j.seps.2024.102095.

[8] N. Matinrad and T. Andersson Granberg, “Optimal pre-dispatch task assignment of volunteers in daily emergency response,” Socio-Economic Planning Sciences, vol. 87, Part B, Art. no. 101589, 2023, doi: 10.1016/j.seps.2023.101589.

[9] P. Di Francesco, P. Lago, and I. Malavolta, “Architecting with microservices: A systematic mapping study,” Journal of Systems and Software, vol. 150, pp. 77–97, 2019, doi: 10.1016/j.jss.2019.01.001.

[10] H. T. Le, L. K. Shar, D. Bianculli, L. C. Briand, and C. D. Nguyen, “Automated reverse engineering of role-based access control policies of web applications,” Journal of Systems and Software, vol. 184, Art. no. 111109, 2022, doi: 10.1016/j.jss.2021.111109.